Automated Network Penetration Testing Using Reinforcement Learning

Majedul Islam Md. Ashraful Alam Md. Rakibul Hassan
Md. Abid Hasan Sazin Sawrave Ahmed
Dr. Muhammad Iqbal Hossain *Dr. Swakkhar Shatabda*
Department of Computer Science and Engineering, BRAC University

Abstract

Due to the massive increase in complexity and frequency of attacks in modern net- work systems, traditional methods of penetration testing often lack scalability and efficiency for repetitive and large-scale assessments. Traditional penetration testing relies on a rule-based approach and model-checking procedures to generate attack graphs. In contrast, RL-based approaches address the challenges of these methods that suit evolving and dynamic network environments . In this paper, we propose a model that simulates and evaluates intelligent attack strategies on virtual network systems using RL. This work contributes to the advancement of penetration testing by applying reinforcement learning to automate the discovery of vulnerabilities and decision-making in dynamic network environments, enabling scalable and adaptive network security evaluation

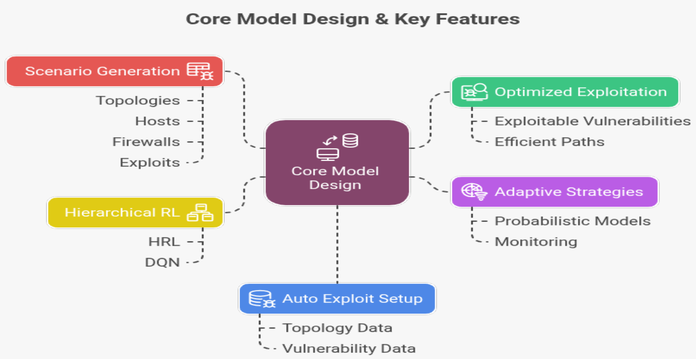
Why Traditional Penetration Testing Falls Short

Challenges	Limitations of Existing Methods
Modern networks are increasingly complex and frequently attacked.	Traditional penetration testing lacks scalability and is inefficient for large, repetitive assessments.
Manual testing is accurate but time-consuming and costly.	Automated tools often generate false positives and require manual supervision.
Existing frameworks focus on known vulnerabilities in controlled setups.	They fail to adapt to real-world, dynamic network environments.
Early RL models (e.g., Q-Tables, DQN) struggle with vast state-action spaces.	These models do not scale effectively in modern infrastructures.

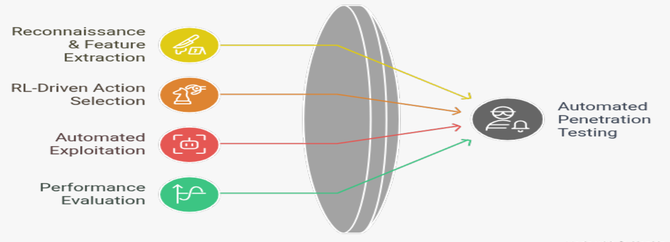
Exploitation Process Objectives

- **TO1:** Selecting the optimal path for exploitation by prioritizing scanned vulnerabilities based on exploitability.
- **TO2:** Filtering false positives (non-exploitable vulnerabilities).
- **TO3:** Choosing the most suitable exploit from tools like Metasploit by mapping vulnerabilities with provided exploits.
- **TO4:** Configuring the exploit and performing the exploitation.
- **TO5:** Adapting a stealthier approach after failed attempts if the system crashes.
- **TO6:** Selecting and configuring suitable payloads based on the target host's characteristics and services.

Core Model Design & Key Features



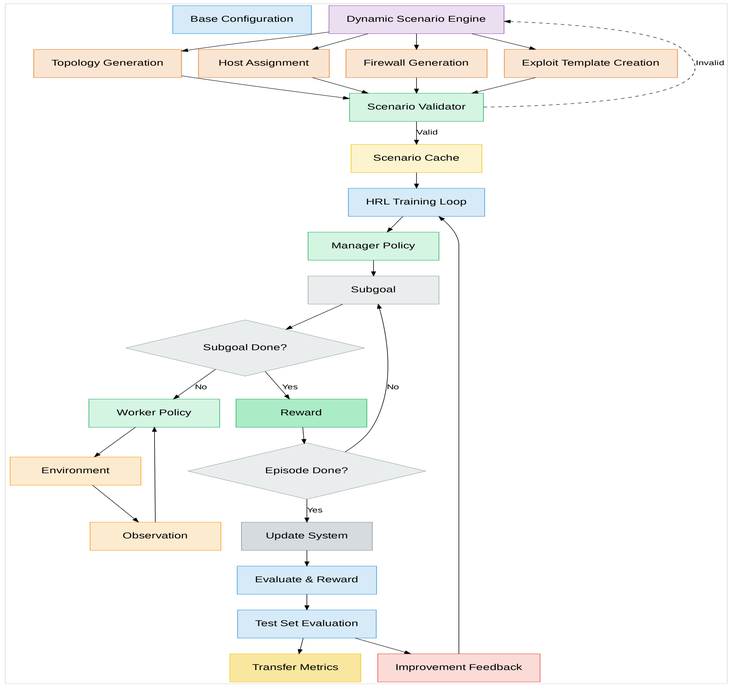
Methodology Overview



Training and Evaluation Pipeline

Proposed Framework Architecture

Component	Summary
Base Configuration	Define network assets, OS types, credentials, and exploit library.
Dynamic Scenario Engine	Automatically generate network topologies, provision hosts, apply firewall rules, and create exploit templates.
Scenario Validator & Cache	Check scenario validity (reachability, logical flows) and store validated setups for reuse.
Core HRL Loop	Manager policy sets strategic goals; Worker policies carry out detailed actions with continuous state observations and reward feedback.
Exploit Execution & Feedback	Deploy exploits via Metasploit (or similar); feed results back into validator and HRL loop to update policies.
Adaptive Evaluation & Metrics	Monitor success rate, time to compromise, and stealth metrics; adapt policies to dynamic network changes.



Challenges Addressed & Future Directions

- Transitioning from theoretical models to practical real-world deployment
- Developing specialized interfaces for state gathering, action execution, and adaptive learning
- Expanding training environment diversity via procedural topology generation and role-based templates
- Integrating temporal dynamics like changing services, firewall rules, and patched exploits during episodes
- Enabling dynamic payload selection and multi-stage attack chain simulation

Results & Analysis

- **Value Increase**: $V(s) \uparrow 14.3\%$ ($0.35 \rightarrow 0.40$) over 100 steps
- **Effective Exploits**: Sequential compromises achieved at $p = 0.8$ success rate
- **Penalty Handling**: Progress maintained despite $r = -0.1$ penalties
- **Efficient Completion**: Objectives met with multiple hosts compromised

Step	$V(s)$	Status
1	0.35	First compromise
100	0.40	Objectives achieved

References

1. Nguyen et al. (2025) PenGym Environment
2. Li et al. (2024) DynPen Framework
3. Janisch et al. (2023) NASimEmu